

BOOTCAMP FRONTEND & REACT · BLOQUE REACT · CLASE 1

Una tienda nueva, desde cero

Por qué existe React, Vite y JSX — todo en TypeScript, desde la primera línea. Tu conocimiento viaja completo; el código se escribe nuevo.

4 h 15 · L-M-V

Proyecto: la tienda del bloque

STEER
RC
LA

EMPIEZA REACT · PROYECTO NUEVO, DESDE CERO

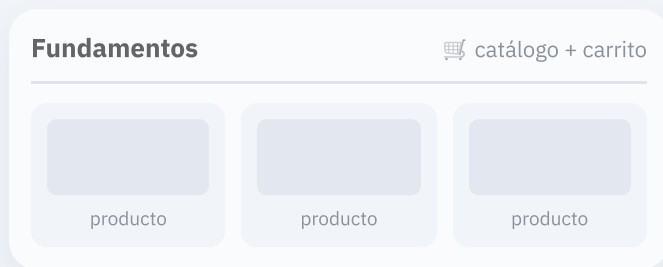
Agenda

- | | | | |
|---|--------|--|-----|
| 1. Punto de partida
la tienda que vamos a construir | 15' | 2. Por qué React
el problema que ya vivimos | 20' |
| 3. Vite: crear el proyecto
en mi pantalla, paso a paso | 35' | 4. JSX
la función que devuelve HTML · 4 errores | 20' |
| 5. La primera pantalla
+ Tailwind entra al build | 30' | 6. El contrato, desde cero
tipos · datos · formato | 35' |
| 7+ FLUJO: componentes y props
material de la clase 2, hasta donde alcance | → 3:50 | C. Cierre + 5 tareas
push demostrado, dudas y anticipo | 25' |

 **Receso de 15 min** entre el bloque 5 y el 6 · verde = **flujo: se corta donde toque y la clase 2 retoma exacto**

DE AQUÍ A LA CLASE 12

La tienda **nueva**



Desde cero: proyecto nuevo, archivos nuevos, todo en TypeScript. Del curso de fundamentos te traes **todo lo que sabes** — y ni un archivo copiado.

Nuestro `main.ts`

los datos

`productos`

`categoriaActiva`

`termino`

TÚ te acuerdas

de llamar a las correctas,
cada vez que algo cambia

las que pintan

`pintarCatalogo()`

`pintarCarrito()`

`pintarTituloCatalogo()`

`marcarCategoriaActiva()`

Si agregas al carrito y olvidas llamar a `pintarCarrito()`, **la página miente**: el dato cambió y la pantalla no. A pequeña escala se vive; **no escala**.

QUÉ ES REACT, EN SERIO

Una sola idea

$$UI = f(\text{datos})$$

Tú cambias el dato — React repinta. Se acabó el acordarse.

IMPERATIVO — FUNDAMENTOS

das órdenes: busca el elemento, bórralo, inserta el HTML. Como un `for` que acumula.

DECLARATIVO — REACT

describes el resultado: "la pantalla es esto, dados estos datos". Como un `reduce`.

TRES ACLARACIONES ANTES DE ESCRIBIR UNA LÍNEA

Qué **NO** es React

No es un framework

Un framework decide todo por ti — como **Spring** en Java. React es una **biblioteca**: una pieza que hace una sola cosa.

No es un lenguaje

Sigue siendo TypeScript. Lo único nuevo de hoy es que una función pueda **devolver HTML**.

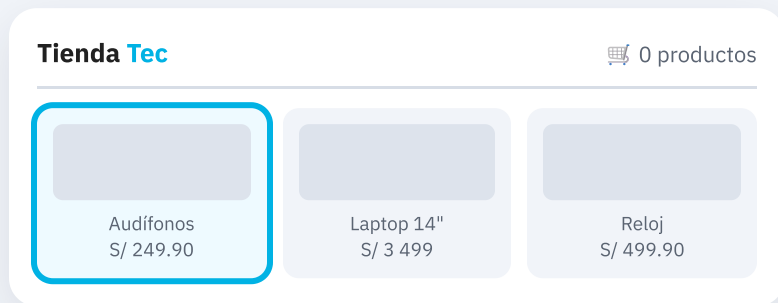
No reemplaza lo aprendido

HTML, CSS, fetch, map, `filter`: todo se usa **tal cual**, dentro de React.

El código se escribe **nuevo**; el conocimiento es el mismo.

LA PALABRA DE LAS DOCE CLASES DE ESTE BLOQUE

Componente



Un pedazo de interfaz **con su propia lógica.**

Una página es un **árbol de componentes** — y cada componente es, literalmente, **una función que devuelve HTML.**

Hoy escribimos el primero.
Cómo se comunican: **clase 2.**

LA LÍNEA DE PRODUCCIÓN, YA ARMADA

Vite

A MANO — CLASE DE TYPESCRIPT

`tsc --watch` traducía

Live Server servía la página

`tsconfig` escrito por nosotros

recarga: la página entera



VITE

servidor + traductor, **un comando**

plantillas: React + TypeScript listos

refresco en caliente: solo la pieza que cambió

build: `tsc` revisa, Vite empaqueta

Ya sabes qué hace cada pieza **porque la armaste a mano**. Vite no es magia: es esa línea, más rápida.

LA ÚNICA PIEZA GENUINAMENTE NUEVA DE HOY

JSX: la función que devuelve HTML

lo que escribes – App.tsx

```
export default function App() {  
  return <h1>Tienda Tec</h1>;  
}
```



lo que el navegador recibe (traducido)

```
export default function App() {  
  return createElement('h1', null,  
    'Tienda Tec');  
}
```

Igual que TypeScript: escribes cómodo, **un traductor convierte**, el navegador recibe JavaScript de siempre.
.tsx = TypeScript + **JSX**.

LA PRIMERA PIEDRA DE LA TIENDA

El contrato, desde cero

tipos.ts

HOY · qué ES un producto — gobierna todo el bloque

datos.ts

HOY · catálogo semilla: 3 productos a mano

formato.ts

HOY · el único que sabe de soles (Intl)

carrito.ts

clase 4 · la lógica pura del carrito

api.ts

clase 7 · la API real + su adaptador

La tienda de fundamentos aprendió esto a golpes (S/ NaN, undefined).

La tienda nueva arranca al revés: **primero el contrato, después los datos.**

CINCO IDEAS, EN EL ORDEN EN QUE LAS VIMOS

Lo de hoy

UI = f(datos)

tú cambias el dato, React repinta —
se acabó el acordarse

Vite

tu `tsc --watch` + Live Server, en
una herramienta con plantillas

JSX

una raíz · `className` · todo se cierra
· datos entre `{ }`

componente

una función que devuelve JSX. Hoy:
App. La comunicación: clase 2

el contrato

`tipos.ts` gobierna; `datos.ts` y
`formato.ts` obedecen

npm run build

sin errores = tu criterio de entrega,
desde hoy y para siempre

CRITERIO DE ENTREGA: NPM RUN BUILD SIN ERRORES + PUSH

5 tareas

FÁCIL

Replica la clase, hasta donde llegamos: Vite + Tailwind, el contrato, el catálogo semilla, el formateador, las tarjetas — con la grabación al lado. Y **bautiza tu tienda**. Es LA tarea.

INTERMEDIO

La página completa: header con el nombre de tu tienda y nav, main con las tarjetas, footer. Todo JSX, todo con utilidades de Tailwind.

INTERMEDIO

La vitrina de tres: muestra tres productos. Si llegamos a ProductCard, úsalo tres veces; si no, copia el article y **comenta qué problema le ves**.

DIFÍCIL

El catálogo crece: amplía datos .ts a 8 productos reales (fuente: dummyjson.com/products) y súmale al contrato un campo opcional descripcion? usado en al menos una tarjeta.

DIFÍCIL

Leer React de verdad: recupera el contador de la plantilla desde react.dev, hazlo funcionar en Practica.tsx, y comenta qué crees que hace ese useState. Lo confirmamos en la clase 4.

Próxima clase — retomamos donde cortamos

La tarjeta se escribe **una vez**

```
<article>...</article> <article>...</article> <article>...</article>
```

```
interface Props {  
  producto: Producto  
}  
  
<ProductCard producto={productos[0]} />
```

El dolor del copy-paste de tu tarea tiene nombre y solución: un **componente** que se escribe una vez y se usa tres, y cada copia recibe su producto por **props**.

La interface Props la sabes leer desde la clase de TypeScript. Trae las tareas hechas y el build en cero.